

Extreme Gradient Boosting

Fecha: 25/01/2025

Autor: Rubén Garrido Hidalgo

XGBoost con Scikit-Learn

En este proyecto, debes implementar XGBoost con Python y Scikit-Learn para resolver un problema de clasificación.

El problema consiste en clasificar a los clientes de dos canales diferentes como clientes de Horeca (Hotel/Retail/Café) o clientes del canal minorista (nominal).

1) Descargamos el conjunto de datos de Wholesale customers desde el siguiente enlace del repositorio de UCI Machine learning:

<https://archive.ics.uci.edu/ml/datasets/Wholesale+customers>

Una vez descargados los datos descomprimos el archivo zip descargado del repositorio de UCI Machine Learning y importamos el archivo wholesale customers data.csv a mi repositorio de jupyter lab.

2) Importamos las librerías de Python necesarias.

```
In [29]: # Importamos las bibliotecas principales
import numpy as np # Para operaciones numéricas
import pandas as pd # Para manipulación de datos

# XGBoost
import xgboost as xgb
from xgboost import XGBClassifier, XGBRegressor # Modelos para clasificación y
from xgboost import plot_importance # Para visualizar la importancia de las car

from sklearn.model_selection import train_test_split # División de datos en con
from sklearn.metrics import accuracy_score, mean_squared_error # Métricas de ev

import matplotlib.pyplot as plt # Para gráficos
import seaborn as sns # Para estilizar mas nuestras gráficas
```

3) Importamos los datos

```
In [9]: ruta_archivo = 'Wholesale customers data.csv'
datos = pd.read_csv(ruta_archivo)
print (datos)
```

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	\
0	2	3	12669	9656	7561	214	2674	
1	2	3	7057	9810	9568	1762	3293	
2	2	3	6353	8808	7684	2405	3516	
3	1	3	13265	1196	4221	6404	507	
4	2	3	22615	5410	7198	3915	1777	
..	...	...	...	...	...	...	...	
435	1	3	29703	12051	16027	13135	182	
436	1	3	39228	1431	764	4510	93	
437	2	3	14531	15488	30243	437	14841	
438	1	3	10290	1981	2232	1038	168	
439	1	3	2787	1698	2510	65	477	

	Delicassen
0	1338
1	1776
2	7844
3	1788
4	5185
..	...
435	2204
436	2346
437	1867
438	2125
439	52

```
[440 rows x 8 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Channel                440 non-null    int64
1   Region                 440 non-null    int64
2   Fresh                  440 non-null    int64
3   Milk                   440 non-null    int64
4   Grocery                 440 non-null    int64
5   Frozen                 440 non-null    int64
6   Detergents_Paper       440 non-null    int64
7   Delicassen             440 non-null    int64
dtypes: int64(8)
memory usage: 27.6 KB
```

4) Realizamos una descripción de los datos (Variables, Volumen, Tipo de las fuentes de datos).

Los datos extraídos del documento "Wholesale customers data.csv" constan de 440 filas y 8 columnas, las cuales indican las ocho variables principales. Los datos hacen referencia a los clientes de un distribuidor mayorista, donde se pueden apreciar los gastos anuales en diferentes aspectos como la leche(milk), los productos frescos (fresh),...

Nuestros datos constan de 8 variables principales

- Channel
- Region
- Fresh
- Milk

- Grocery
- Frozen
- Detergents_Paper
- Delicassen

Hay diferentes tipos de variables, pues mientras channel y region son variables nominales donde se expresan un número entero en función de la localización(región), o el canal de venta(chanel). Las demás variables son continuas y se expresan con números enteros sin decimales. Podemos implementar la funcion datos.info() para ver más caracetrísticas de los datos, pero nos interesa para ver el tipo de datos el cual será int64

In [10]: `datos.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Channel                440 non-null    int64
1   Region                 440 non-null    int64
2   Fresh                  440 non-null    int64
3   Milk                   440 non-null    int64
4   Grocery                440 non-null    int64
5   Frozen                 440 non-null    int64
6   Detergents_Paper       440 non-null    int64
7   Delicassen             440 non-null    int64
dtypes: int64(8)
memory usage: 27.6 KB
```

5) Exploración de los datos (preview, summary del dataframe (n. de datos nulos, tipo de las variables, resumen de la estadística descriptiva, valores perdidos (missing))

```
In [25]: #Preview: mostramos solo algunas de las primeras filas de los datos.
print("Preview:")
print(datos.head())

#Summary: número de datos nulos y tipo de las variables.
print("Summary del dataframe")
print(datos.info())

#Estadística descriptiva: calculamos la media, varianza, máximos,, mínimos y cua
#de las variables(columnas).
print("Estadística descriptiva:")
print(datos.describe())

# Valores perdidos
print("\nValores nulos por columna:")
print(datos.isnull().sum())
```

Preview:

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	2	3	12669	9656	7561	214	2674	1338
1	2	3	7057	9810	9568	1762	3293	1776
2	2	3	6353	8808	7684	2405	3516	7844
3	1	3	13265	1196	4221	6404	507	1788
4	2	3	22615	5410	7198	3915	1777	5185

Summary del dataframe

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 440 entries, 0 to 439

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	Channel	440 non-null	int64
1	Region	440 non-null	int64
2	Fresh	440 non-null	int64
3	Milk	440 non-null	int64
4	Grocery	440 non-null	int64
5	Frozen	440 non-null	int64
6	Detergents_Paper	440 non-null	int64
7	Delicassen	440 non-null	int64

dtypes: int64(8)

memory usage: 27.6 KB

None

Estadística descriptiva:

	Channel	Region	Fresh	Milk	Grocery \
count	440.000000	440.000000	440.000000	440.000000	440.000000
mean	1.322727	2.543182	12000.297727	5796.265909	7951.277273
std	0.468052	0.774272	12647.328865	7380.377175	9503.162829
min	1.000000	1.000000	3.000000	55.000000	3.000000
25%	1.000000	2.000000	3127.750000	1533.000000	2153.000000
50%	1.000000	3.000000	8504.000000	3627.000000	4755.500000
75%	2.000000	3.000000	16933.750000	7190.250000	10655.750000
max	2.000000	3.000000	112151.000000	73498.000000	92780.000000

	Frozen	Detergents_Paper	Delicassen
count	440.000000	440.000000	440.000000
mean	3071.931818	2881.493182	1524.870455
std	4854.673333	4767.854448	2820.105937
min	25.000000	3.000000	3.000000
25%	742.250000	256.750000	408.250000
50%	1526.000000	816.500000	965.500000
75%	3554.250000	3922.000000	1820.250000
max	60869.000000	40827.000000	47943.000000

Valores nulos por columna:

Channel	0
Region	0
Fresh	0
Milk	0
Grocery	0
Frozen	0
Detergents_Paper	0
Delicassen	0

dtype: int64

6) ¿Cuál es la variable target (y) y las variables que la explican (feature vector - X)?

La variable (target) o variable a explicar es "channel", pues nuestro proyecto consiste en implementar el algoritmo de aprendizaje automático XGBoosting para clasificar a los clientes de este proveedor en dos canales diferentes Horeca(Hotel,Retail y café) o minorista (Retail). Las otras siete variables serán las variables que explicarán el tipo de cliente que tiene este mayorista.

- Region
- Fresh
- Milk
- Grocery
- Frozen
- Detergents_Paper
- Delicassen

7) Dividimos el conjunto de datos en entrenamiento (train) y prueba (test).

```
In [31]: import numpy as np # Para operaciones numéricas
import pandas as pd # Para manipulación de datos

# XGBoost
import xgboost as xgb
from xgboost import XGBClassifier, XGBRegressor # Modelos para clasificación y
from xgboost import plot_importance # Para visualizar la importancia de las car

from sklearn.model_selection import train_test_split # División de datos en con
from sklearn.metrics import accuracy_score, mean_squared_error # Métricas de ev

import matplotlib.pyplot as plt # Para gráficos

#Datos
ruta_archivo = 'Wholesale customers data.csv'
datos = pd.read_csv(ruta_archivo)

#Asignamos el conjunto de datos a la variable caractreística y las demás
y = datos['Channel'] -1 # Para que XGBoost trabaje con las clases de [0,1]
X = datos.drop(columns=['Channel'])

#División del conjunto de entremiento con un 80% de los datos empleados en el en
# y un 20% empelados en la prueba del modelo
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Crear el clasificador
model = xgb.XGBClassifier()

# Entrenar el modelo con los datos de entrenamiento
model.fit(X_train, y_train)
```

Out[31]:

```

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               gamma=None, grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=None, max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None,
               max_depth=None, max_leaves=None, min_child_weight=None, missing=None,
               monotone_constraints=None, multi_strategy=None, n_estimators=None,
               n_jobs=None, num_parallel_tree=None, random_state=None, reg_alpha=None,
               reg_lambda=None, sampling_method=None, scale_pos_weight=None, subsample=None,
               tree_method=None, validate_parameters=None, verbosity=None)

```

Este resultado es la salida cuando imprimes un objeto XGBClassifier sin configurarlo explícitamente o cuando el modelo se crea pero no se han definido valores para los hiperparámetros. En este caso, los valores predeterminados de muchos hiperparámetros están configurados como None, lo que significa que XGBoost utilizará automáticamente los valores predeterminados internos.

8) ¿Cuáles son los hiperparámetros? ¿Qué valores tienen?

In [39]: *#Para ver los hiperparámetros del modelo podemos implementar el siguiente código*

```

#Lista de hiperparámetros
parametros = model.get_params()
nombre_parametros = list(parametros.keys())

print("Estos son los hiperparámetros del modelo")
print(nombre_parametros)

```

Estos son los hiperparámetros del modelo

```

['objective', 'base_score', 'booster', 'callbacks', 'colsample_bylevel', 'colsample_bynode', 'colsample_bytree', 'device', 'early_stopping_rounds', 'enable_categorical', 'eval_metric', 'feature_types', 'gamma', 'grow_policy', 'importance_type', 'interaction_constraints', 'learning_rate', 'max_bin', 'max_cat_threshold', 'max_cat_to_onehot', 'max_delta_step', 'max_depth', 'max_leaves', 'min_child_weight', 'missing', 'monotone_constraints', 'multi_strategy', 'n_estimators', 'n_jobs', 'num_parallel_tree', 'random_state', 'reg_alpha', 'reg_lambda', 'sampling_method', 'scale_pos_weight', 'subsample', 'tree_method', 'validate_parameters', 'verbosity']

```

In [40]:

```
print("Estos son los valores de los hiperparámetros")
#model = XGBClassifier()
print(model.get_params())
```

Estos son los valores de los hiperparámetros

```
{'objective': 'binary:logistic', 'base_score': None, 'booster': None, 'callbacks': None, 'colsample_bylevel': None, 'colsample_bynode': None, 'colsample_bytree': None, 'device': None, 'early_stopping_rounds': None, 'enable_categorical': False, 'eval_metric': None, 'feature_types': None, 'gamma': None, 'grow_policy': None, 'importance_type': None, 'interaction_constraints': None, 'learning_rate': None, 'max_bin': None, 'max_cat_threshold': None, 'max_cat_to_onehot': None, 'max_delta_step': None, 'max_depth': None, 'max_leaves': None, 'min_child_weight': None, 'missing': nan, 'monotone_constraints': None, 'multi_strategy': None, 'n_estimators': None, 'n_jobs': None, 'num_parallel_tree': None, 'random_state': None, 'reg_alpha': None, 'reg_lambda': None, 'sampling_method': None, 'scale_pos_weight': None, 'subsample': None, 'tree_method': None, 'validate_parameters': None, 'verbosity': None}
```

9) ¿Cuáles son los hiperparámetros?

Los hiperparámetros que podemos observar tras entrenar nuestro algoritmo de XGBoost son:

- **objective:** función objetivo.
- **base_score:** valor inicial de predicción antes del entrenamiento.
- **booster:** algoritmo base empleado para el boosting.
- **callbacks:** Permite usar funciones que controlan el proceso de entrenamiento, como la detención temprana.
- **colsample_bylevel:** Proporción de características que se usan en cada nivel de un árbol.
- **colsample_bynode:** Proporción de características que se usan en cada nodo de un árbol.
- **colsample_bytree:** Fracción de características usadas en cada árbol.
- **device:** El dispositivo utilizado para entrenar el modelo.
- **early_stopping_rounds:** Número de rondas sin mejora para detener el entrenamiento anticipadamente.
- **enable_categorical:** Habilita el manejo de variables categóricas directamente en el modelo.
- **eval_metric:** métrica de evaluación del modelo:
- **feature_types:** Permite especificar el tipo de cada característica (por ejemplo, numérica o categórica).
- **gamma:** restricción mínima de pérdida para la división de nodos.
- **grow_policy:** Controla la estrategia de crecimiento del árbol.
- **importance_type:** Método para calcular la importancia de las características.
- **interaction_constraints:** Impone restricciones sobre qué características pueden interactuar entre sí en el árbol.
- **learning_rate:** tasa de aprendizaje del algoritmo.
- **max_bin:** Controla el número máximo de bins (intervalos) para discretizar los datos continuos durante la construcción del árbol.
- **max_delta_step:** Controla el tamaño del paso de actualización en el proceso de optimización, útil para evitar inestabilidad en el entrenamiento.
- **max_depth:** profundidad máxima de los árboles.
- **max_leaves:** Número máximo de hojas en el árbol.
- **min_child_weight:** Peso mínimo de las instancias en una hoja para que se divida.

- missing: Valor para representar los valores faltantes.
- monotone_constraints: Restricciones para garantizar que el modelo sea monótono con respecto a ciertas características.
- multi_strategy: Estrategia para clasificación multiclase.
- n_estimators: numero de árboles empleados en el boosting.
- n_jobs: Número de hilos para la ejecución en paralelo durante el entrenamiento.
- num_parallel_tree: Número de árboles construidos en paralelo.
- random_state: Semilla para la generación de números aleatorios.
- reg_alpha: Regularización L1.
- reg_lambda: Regularización L2.
- sampling_method: Método de muestreo (como sobremuestreo o submuestreo).
- scale_pos_weight: Peso de la clase positiva en clasificación desbalanceada.
- subsample: porcentaje usado en cada iteración.
- tree_method: Define el método de construcción de árboles.
- validate_parameters: Controla si XGBoost valida los parámetros del modelo al inicializarlo.
- verbosity: Controla el nivel de detalle de la salida en consola durante el entrenamiento (valores entre 0 y 3).

¿Qué valores tienen?

```
In [4]: print("Estos son los valores de los hiperparámetros")
#model = XGBClassifier()
print(model.get_params())
```

Estos son los valores de los hiperparámetros

```
{'objective': 'binary:logistic', 'base_score': None, 'booster': None, 'callbacks': None, 'colsample_bylevel': None, 'colsample_bynode': None, 'colsample_bytree': None, 'device': None, 'early_stopping_rounds': None, 'enable_categorical': False, 'eval_metric': None, 'feature_types': None, 'gamma': None, 'grow_policy': None, 'importance_type': None, 'interaction_constraints': None, 'learning_rate': None, 'max_bin': None, 'max_cat_threshold': None, 'max_cat_to_onehot': None, 'max_delta_step': None, 'max_depth': None, 'max_leaves': None, 'min_child_weight': None, 'missing': nan, 'monotone_constraints': None, 'multi_strategy': None, 'n_estimators': None, 'n_jobs': None, 'num_parallel_tree': None, 'random_state': None, 'reg_alpha': None, 'reg_lambda': None, 'sampling_method': None, 'scale_pos_weight': None, 'subsample': None, 'tree_method': None, 'validate_parameters': None, 'verbosity': None}
```

Dichos valores al ser en la mayoría de casos "None" implica que toman los valores predeterminados del algoritmo.

10) Realizamos las predicciones

```
In [6]: import numpy as np # Para operaciones numéricas
import pandas as pd # Para manipulación de datos

# XGBoost
import xgboost as xgb
from xgboost import XGBClassifier, XGBRegressor # Modelos para clasificación y
from xgboost import plot_importance # Para visualizar la importancia de las car
from sklearn.model_selection import train_test_split # División de datos en con
```



```

from sklearn.metrics import accuracy_score, mean_squared_error # Métricas de ev

import matplotlib.pyplot as plt # Para gráficos

#Datos
ruta_archivo = 'Wholesale customers data.csv'
datos = pd.read_csv(ruta_archivo)

#Asignamos el conjunto de datos a la variable característica y las demás
y = datos['Channel'] - 1 # Para que XGBoost trabaje con las clases de [0,1]
X = datos.drop(columns=['Channel'])

#División del conjunto de entrenamiento con un 80% de los datos empleados en el en
# y un 20% empleados en la prueba del modelo
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Crear el clasificador
model = xgb.XGBClassifier()

# Entrenar el modelo con los datos de entrenamiento
model.fit(X_train, y_train)

## Predicciones del modelo
y_pred = model.predict(X_test)
print("Realizamos las predicciones del modelo:")
print(y_pred)

```

Realizamos las predicciones del modelo:

```

[0 0 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0
 1 0 0 0 0 0 0 1 0 1 1 1 0 0 0 0 0 1 1 1 0 1 1 0 0 0 1 0 1 1 0 0 0 1 0 0 0
 0 0 0 0 1 0 0 0 0 1 0 1 0 0]

```

11) Cálculo del accuracy

```

In [44]: accuracy = accuracy_score(y_test, y_pred)
print(f"Precisión del modelo: {accuracy:.2f}")

```

Precisión del modelo: 0.92

12) Realizamos la cross-validation usando XGBoost.

```

In [2]: import numpy as np # Para operaciones numéricas
import pandas as pd # Para manipulación de datos

import warnings
warnings.filterwarnings("ignore", category=UserWarning)

# XGBoost
import xgboost as xgb
from xgboost import XGBClassifier, XGBRegressor # Modelos para clasificación y
from xgboost import plot_importance # Para visualizar la importancia de las car

from sklearn.model_selection import train_test_split # División de datos en con
from sklearn.metrics import accuracy_score, mean_squared_error # Métricas de ev

import matplotlib.pyplot as plt # Para gráficos

#Datos
ruta_archivo = 'Wholesale customers data.csv'
datos = pd.read_csv(ruta_archivo)

```

```

#Asignamos el conjunto de datos a la variable característica y las demás
y = datos['Channel'] -1 # Para que XGBoost trabaje con las clases de [0,1]
X = datos.drop(columns=['Channel'])

#División del conjunto de entrenamiento con un 80% de los datos empleados en el en
# y un 20% empleados en la prueba del modelo
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Crear el clasificador
model = xgb.XGBClassifier()

# Entrenar el modelo con los datos de entrenamiento
model.fit(X_train, y_train)

## Predicciones del modelo
y_pred = model.predict(X_test)
#print("Realizamos las predicciones del modelo:")

#Veo que parámetros tiene la función xgb.cv
#help(xgb.cv)

# Nuestra función xgb.cv requerirá de dos parámetros principales, los datos de e

#Datos de entrenamientos
datos_entre = xgb.DMatrix(X_train, label=y_train)

#Hiperparámetros
hiper_model = model.get_params()

#Realizamos la validación cruzada que viene implementada como una función en el
res_cross_validation = xgb.cv(params=hiper_model, dtrain=datos_entre)
print(res_cross_validation)

```

	train-logloss-mean	train-logloss-std	test-logloss-mean	test-logloss-std
0	0.449120	0.007625	0.473523	0.017871
1	0.343210	0.005158	0.388188	0.011772
2	0.273896	0.007078	0.334198	0.009394
3	0.223452	0.007335	0.294628	0.006274
4	0.186798	0.006724	0.274073	0.003450
5	0.159939	0.006896	0.260002	0.001263
6	0.139645	0.006769	0.252235	0.002502
7	0.123350	0.006428	0.250002	0.004315
8	0.110349	0.006920	0.246980	0.005249
9	0.100335	0.006625	0.249380	0.004621

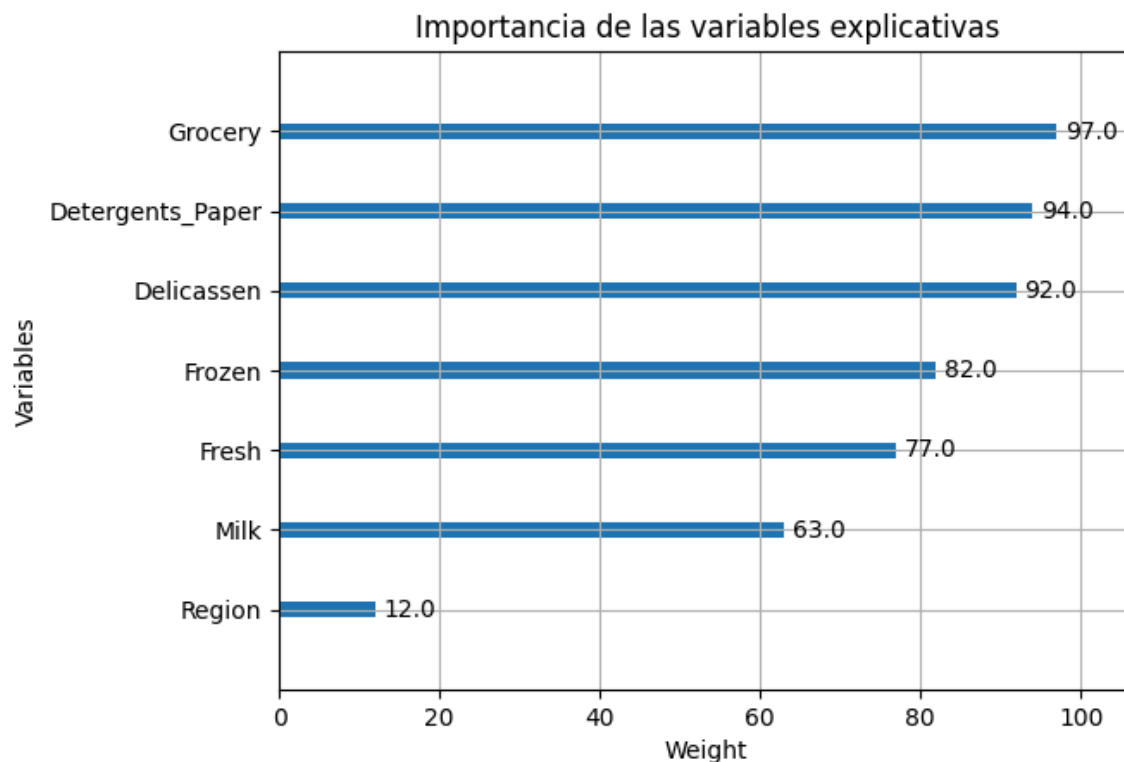
13) Representación gráfica de la importancia de las variables explicativas del modelo XGBoost.

```

In [4]: #Vamos a emplear la función plt.figure incluida en el paquete xgboost
plt.figure(figsize=(10, 8))
repre = plot_importance(model, importance_type='weight', max_num_features=10) #w
#que como sabemos emplea un Random Forest (compendio de árboles de decisión)
repre.set_ylabel("Variables")
#Precisión
repre.set_xlabel("Weight")
plt.title("Importancia de las variables explicativas")
plt.show()

```

<Figure size 1000x800 with 0 Axes>



14) Resultados y conclusión.

En cuanto a los resultados de la implementación del algoritmo XGBoost podemos ver que el algoritmo clasifica bien los datos. Esto lo podemos observar viendo el valor de la precisión(accuracy) que por definición mide las predicciones bien hechas con respecto al número total de datos y al ser 0.92 indica que el 92% de las predicciones son correctas.

También podemos observar el resultado de la validación cruzada, pues los valores de los diferentes hiperparámetros "train-logloss-mean", "train-logloss-std", "test-logloss-mean" y "test-logloss-std" decrecen a medida que aumentan las iteraciones, en este caso hemos definido nueve iteraciones. Lo cual nos permite deducir que nuestro modelo reduce el error en las predicciones, es decir predice mejor y por lo tanto se adapta mejor a nuestros datos.

Por último, debemos evaluar la última representación gráfica, la cual nos indica que siguiendo la métrica weight, que la variable "Grocery", "Detergents_paper" y "Delicassen" son las variables que más se emplearon en los nodos de decisión de los árboles del modelo. Además, indica que al tener más peso tendrán más impacto sobre la variable que queremos explicar.

Conclusión:

Al implementar el algoritmo XGBoost para poder clasificar a los clientes de un proveedor mayorista en clientes de Horeca o de canal minorista hemos podido ver que basándonos en los datos extraídos del archivo CSV y tras entrenar nuestro algoritmo de aprendizaje automático que con un acierto en las predicciones de un 92%, el 70.4% de los clientes son Hoteles, restaurantes y cafeterías. Y un 29.54% son minoristas. También hemos podido deducir que las variables que más influyen a la hora de predecir esto son

"Grocery", "Detergents_paper" y "Delicassen", pues son las que contienen mayor peso. Por lo tanto, la compra de alimentos, productos de delicatessen y detergente y papel son los que tienen más importancia a la hora de predecir el tipo de cliente que tiene el mayorista.

15) Predicción:

Donde 0 corresponde a la clase Horeca (Hoteles/ Restaurantes y Cafeterías) y 1 a la clase Minorista (Retail)

```
[0 0 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 1  
1 0 0 0 0 0 1 1 1 0 1 1 0 0 0 1 0 1 1 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0]
```

$62 < - 0, 62/88 = 0.704 = 70.4\%$

$26 < - 1, 26/88 = 0.2954 = 29.54\%$